

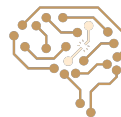
Markov Decision Process (MDP)

Arash Marioriyad

31 Khordad 1405

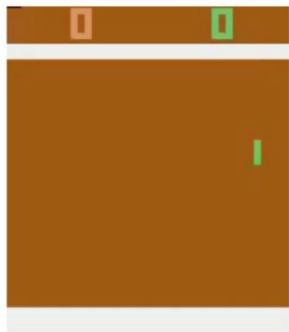
CE 417-Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188 &
Sharif UT CE417

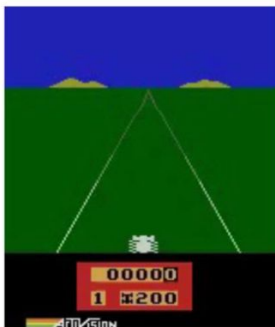


Example of Reinforcement Learning

- 2013: Playing Atari Games



Pong



Enduro



Beamrider



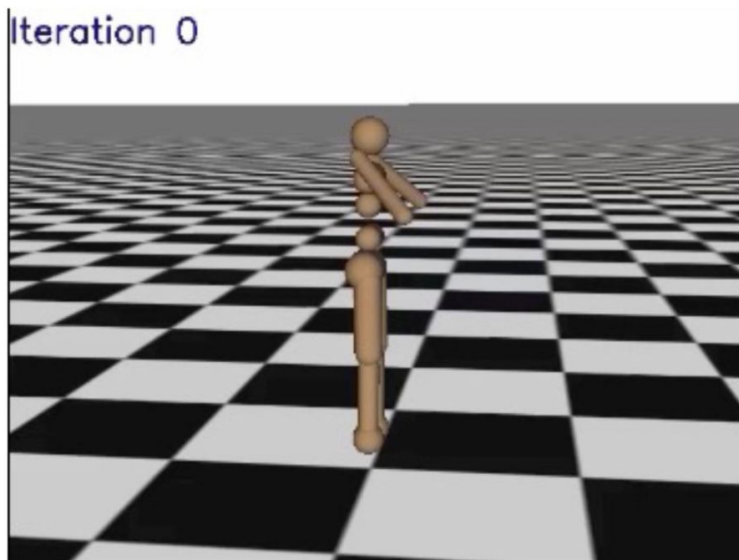
Q*bert

[Human-level control through deep reinforcement learning. Mnih et al. Nature 2015]



Example of Reinforcement Learning

- 2015: Locomotion from Trial and Error



[Trust Region Policy Optimization. Schulman et al. ICLR 2015]

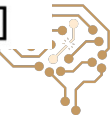


Example of Reinforcement Learning

- 2016: Playing Go (and beating human champion)

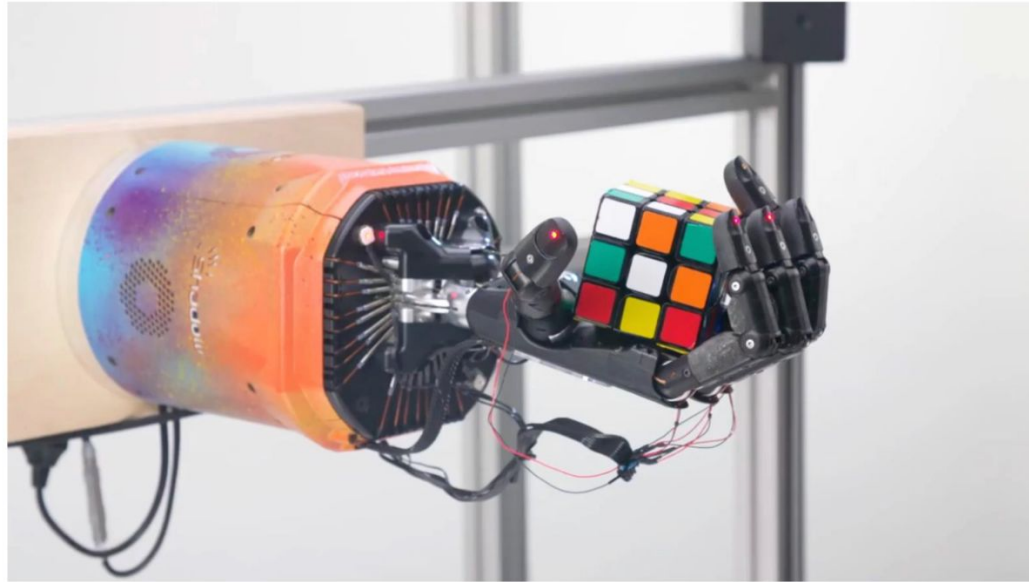


[Mastering the game of Go with deep neural networks and tree search. Silver et al. Nature 2016]



Example of Reinforcement Learning

- 2019: Robot manipulation

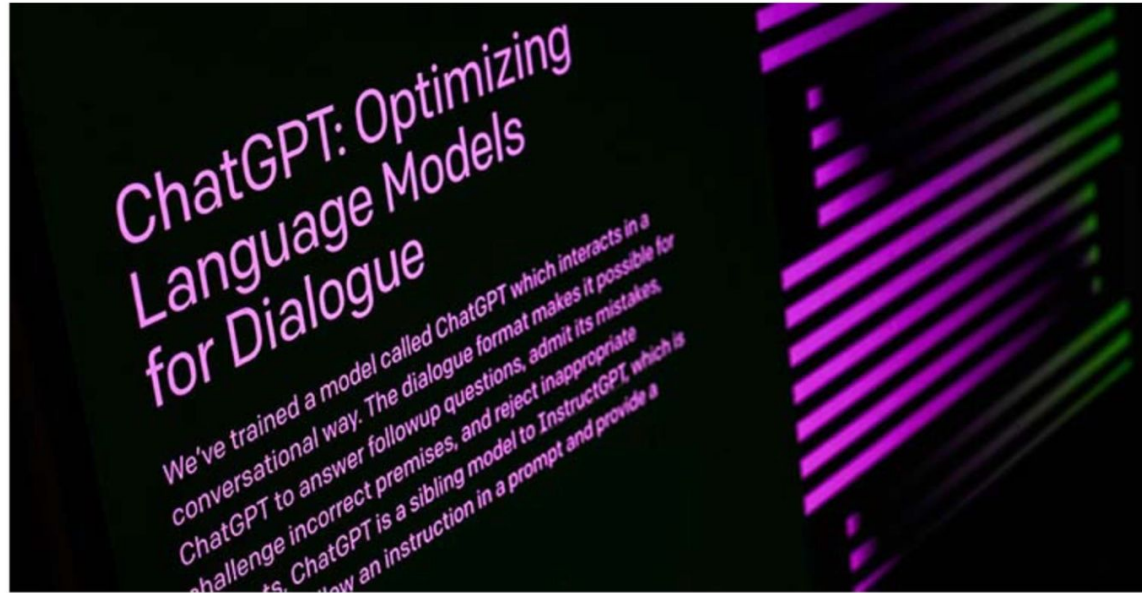


[Solving Rubik's cube with a robot hand. OpenAI. 2019]

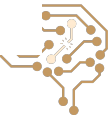


Example of Reinforcement Learning

- 2022: Training Language Assistants with Human Feedback



[Aligning language models to follow instructions. Ouyang et al. 2022]



RL vs Search

- **Problem Scope**

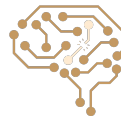
- Search: Solves a single, well-defined problem instance, with a single start state
- Reinforcement Learning: Learns a general policy for all states

- **Model Knowledge**

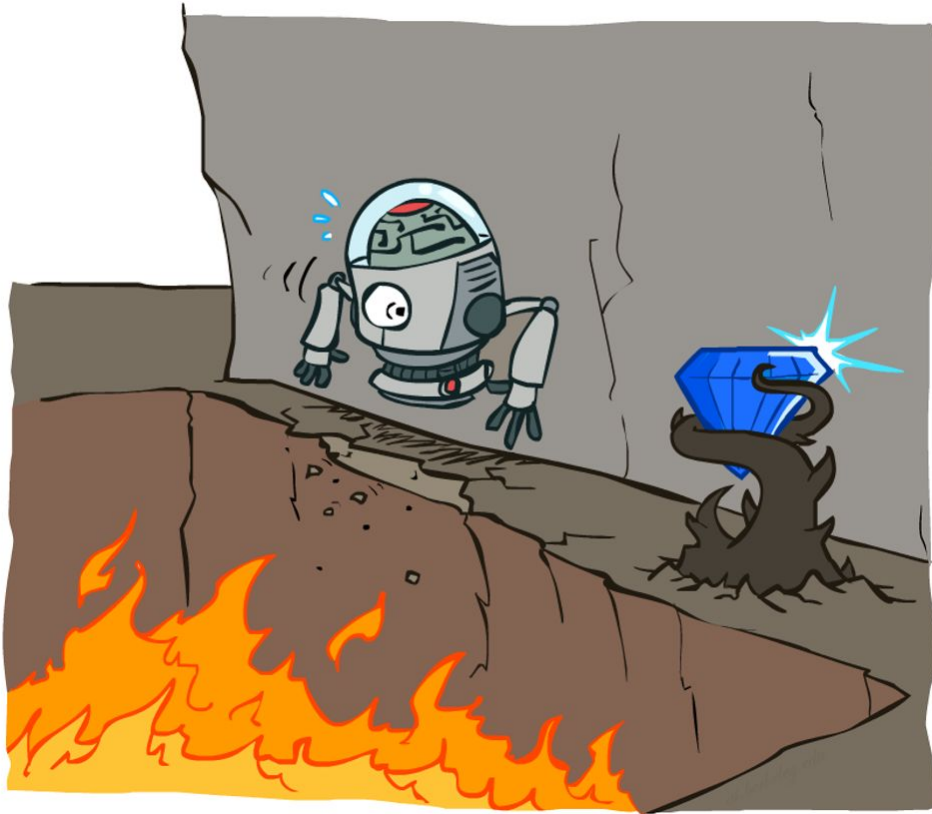
- Search: Assumes a known and static model of the environment.
- Reinforcement Learning: Model may be unknown; learned via interaction.

- **Uncertainty**

- Search: Typically deterministic (or explicitly modeled).
- Reinforcement Learning: Naturally handles stochastic dynamics and rewards



Today: Non Deterministic Search



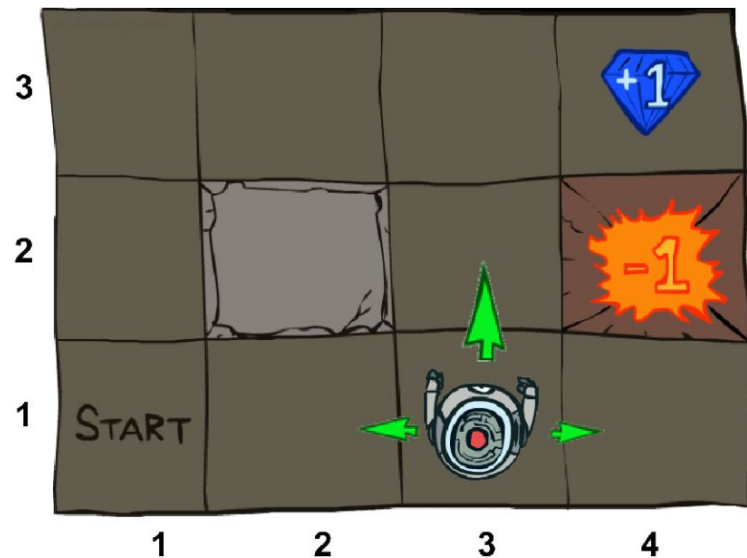
Types of Problems

- ✓ Search (dfs, bfs)
- ✓ Search with Cost (ucs, A*)
- ✓ Local Search (Random Walk, Hill Climbing, Beam Search, ...)
- ✓ Constraint Satisfaction Problems (Backtracking)
- ✓ Search with Other Agents (Minimax)
- ? Non-Deterministic Search



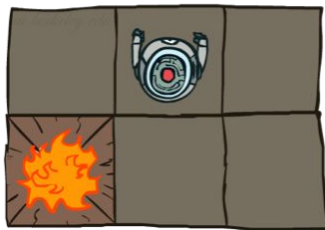
Example: Grid World

- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Noisy movement: actions do not always go as planned
 - 80% of the time, the action North takes the agent North (if there is no wall there)
 - 10% of the time, North takes the agent West; 10% East
 - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
 - Small "living" reward each step (can be negative)
 - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards

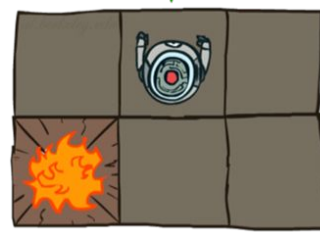
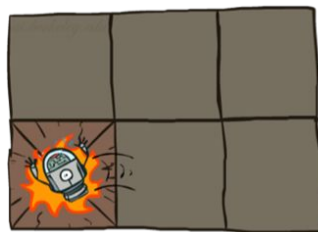
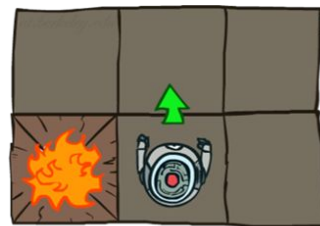


Grid World Actions

Deterministic Grid World

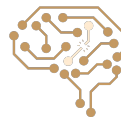
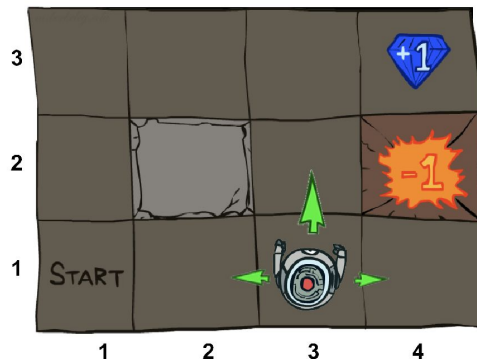


Stochastic Grid World

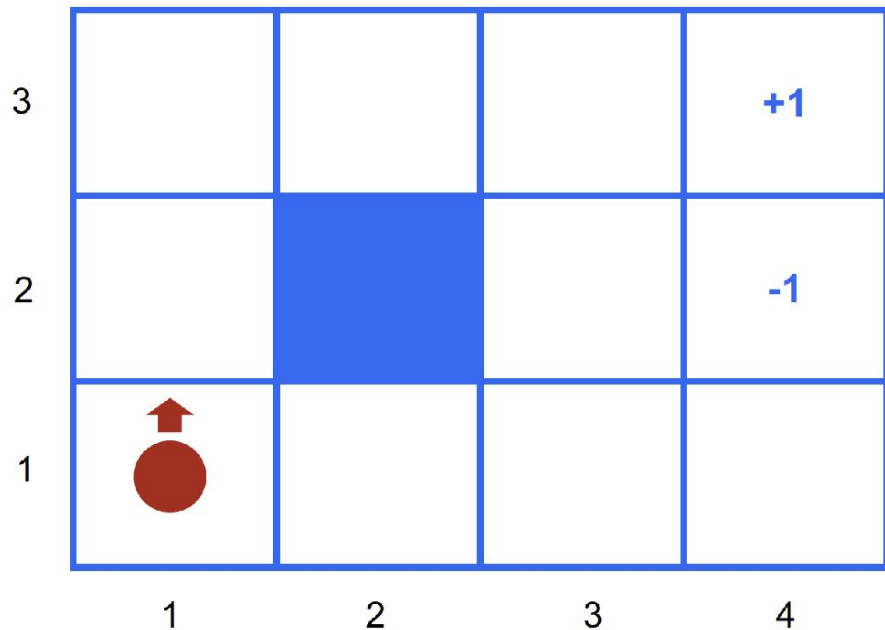


Formulation: Markov Decision Processes

- An MDP is defined by:
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s' | s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
 - A start state
 - Maybe a terminal state



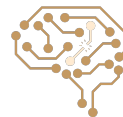
Grid World Example



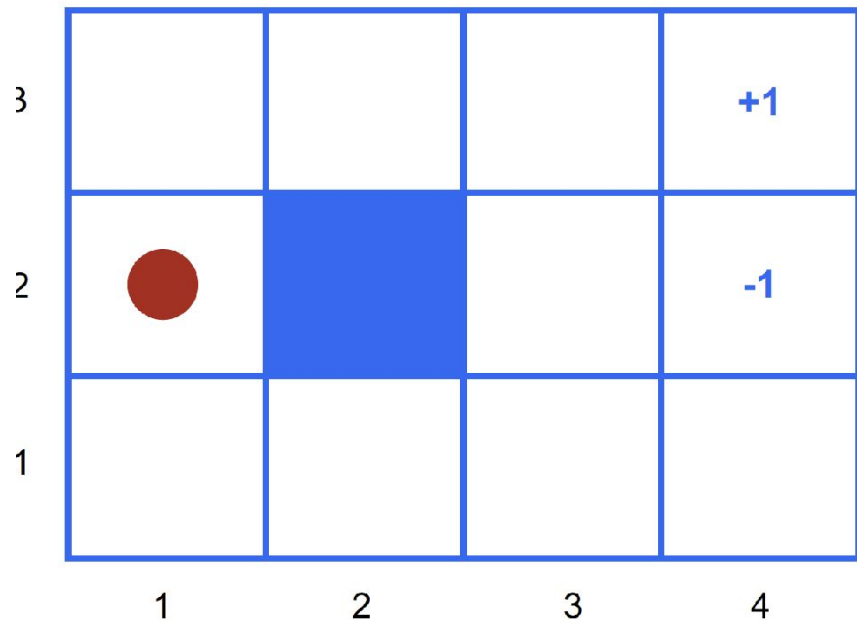
s	a	s'	R
(1,1)	north		

$T(s, a, s')$:

- $T((1,1), \text{north}, (2,1)) = 0.8$
- $T((1,1), \text{north}, (1,2)) = 0.1$
- $T((1,1), \text{north}, (1,1)) = 0.1$



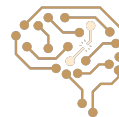
Grid World Example



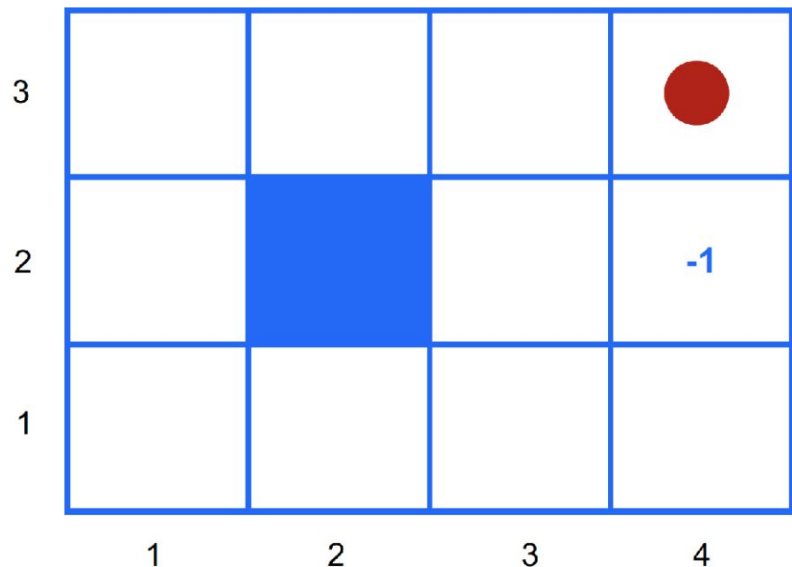
s	a	s'	R
(1,1)	north	(2,1)	-0.1

$R(s, a, s')$:

$R((1,1), \text{north}, (2,1)) = -0.1$



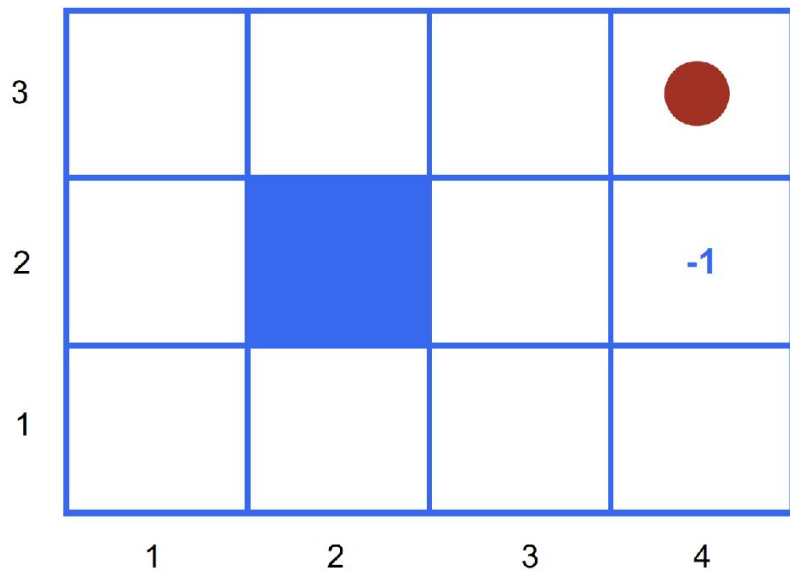
Grid World Example



Q: What's missing from the state transition table?

s	a	s'	R
(1,1)	north	(2,1)	-0.1
(1,1)	north	(1,2)	-0.1
(2,1)	north	(3,1)	-0.1
(1,2)	west	(1,1)	-0.1
(3,1)	east	(2,1)	-0.1
(3,1)	east	(3,2)	-0.1
(3,2)	east	(3,3)	-0.1
(1,3)	west	(1,2)	-0.1
(1,3)	west	(2,3)	-0.1
(2,3)	west	(1,3)	-0.1
(2,3)	west	(3,3)	-0.1
(1,4)	south	(1,3)	-0.1
(3,3)	east	(3,4)	-0.1
(3,3)	east	(2,3)	-0.1
(3,4)	exit	gameover	1.0

Grid World Example



A: All the same-state transitions

s	a	s'	R
(1,1)	north	(1,1)	-0.1
(2,1)	north	(2,1)	-0.1
(1,2)	west	(1,2)	-0.1
(3,1)	east	(3,1)	-0.1
(3,2)	east	(3,2)	-0.1
(1,3)	west	(1,3)	-0.1
(2,3)	west	(2,3)	-0.1
(1,4)	south	(1,4)	-0.1
(3,3)	east	(3,3)	-0.1

Formulation: What is Markov about MDPs?

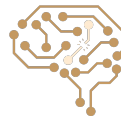
- ▶ “Markov” generally means that given the present state, the future and the past are independent
- ▶ For Markov decision processes, “Markov” means action outcomes depend only on the current state

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0)$$

=

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

- ▶ This is just like search, where the successor function could only depend on the current state (not the history)



Formulation: How can we represent agent movements?

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

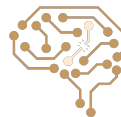
$$s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2, \dots$$



Formulation: What is the purpose of the agent?

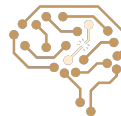
maximize it's reward across all timesteps (**Return**):

$$U([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + R(s_2, a_2, s_3) + \dots$$

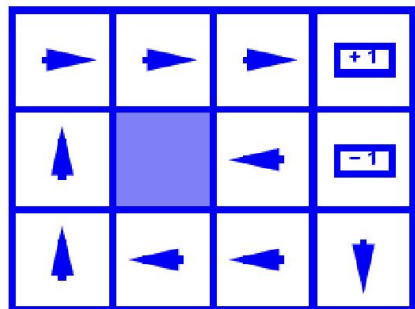


Optimal Policy

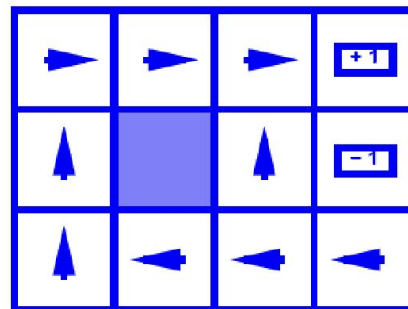
- In deterministic single-agent search problems, we wanted an optimal **plan**, or sequence of actions, from start to a goal
- For MDPs, we want an optimal **policy** $\pi^*: S \rightarrow A$
 - A policy π gives an action for each state
 - An optimal policy is one that **maximizes expected utility** if followed
- Expectimax didn't compute entire policies
 - It computed the action for a single state only



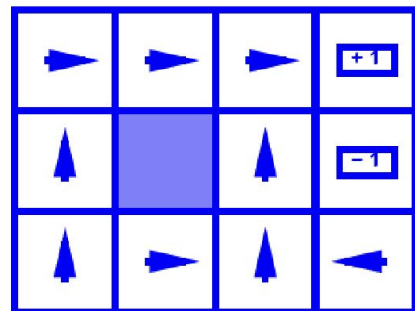
Optimal Policy Example



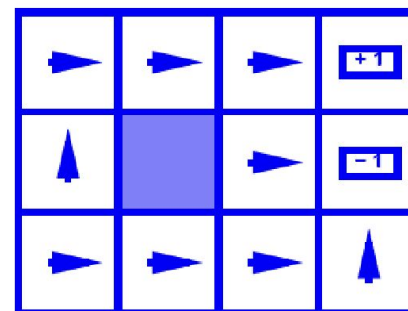
$$R(s) = -0.01$$



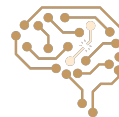
$$R(s) = -0.03$$



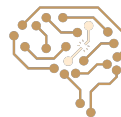
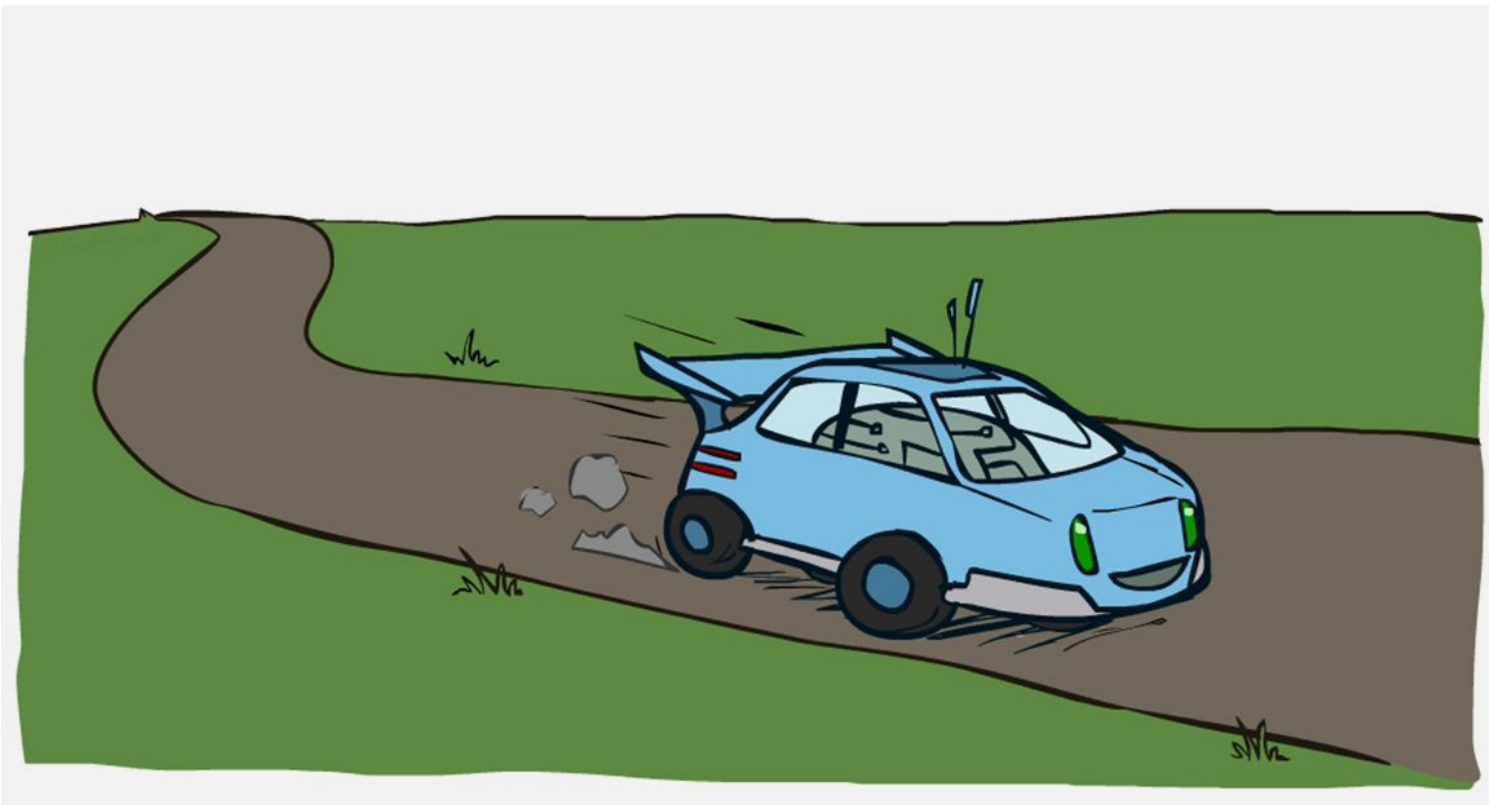
$$R(s) = -0.4$$



$$R(s) = -2.0$$

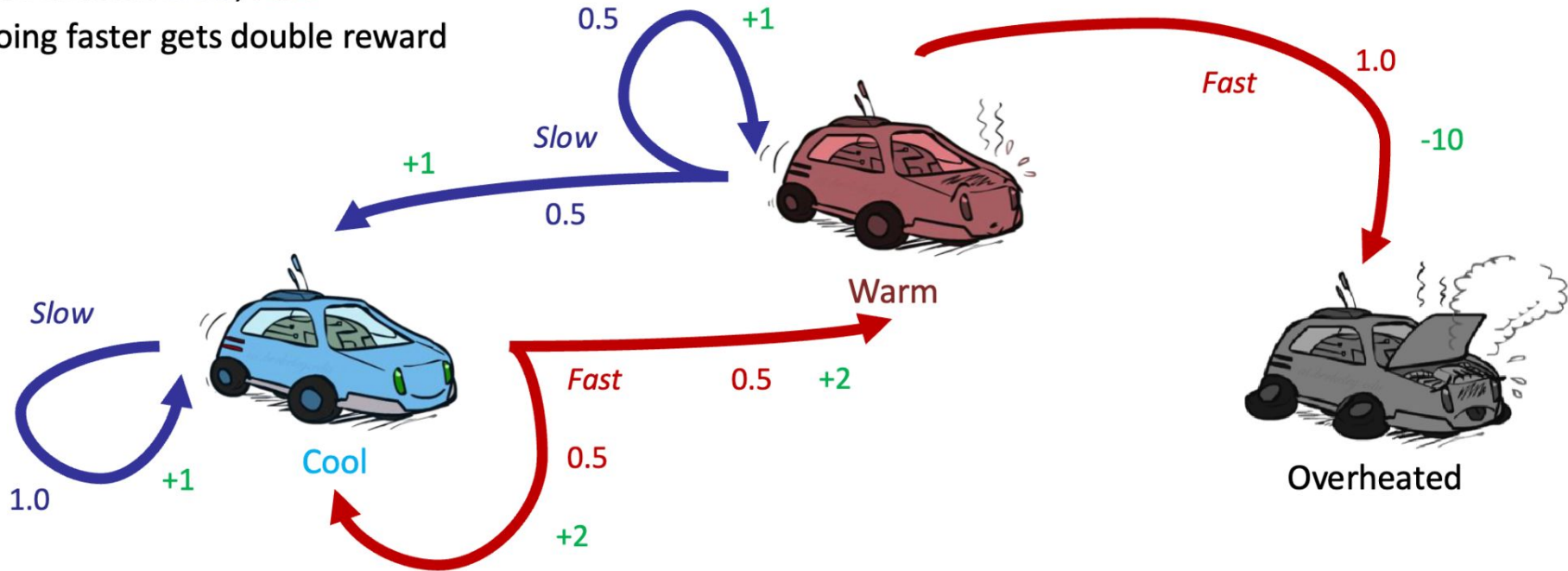


Formulation Example: Racing



Formulation Example: Racing

- A robot car wants to travel far, quickly
- Three states: **Cool**, **Warm**, Overheated
- Two actions: **Slow**, **Fast**
- Going faster gets double reward



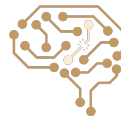
Formulation Example: Racing

- **Transition Function:** $T(s, a, s')$

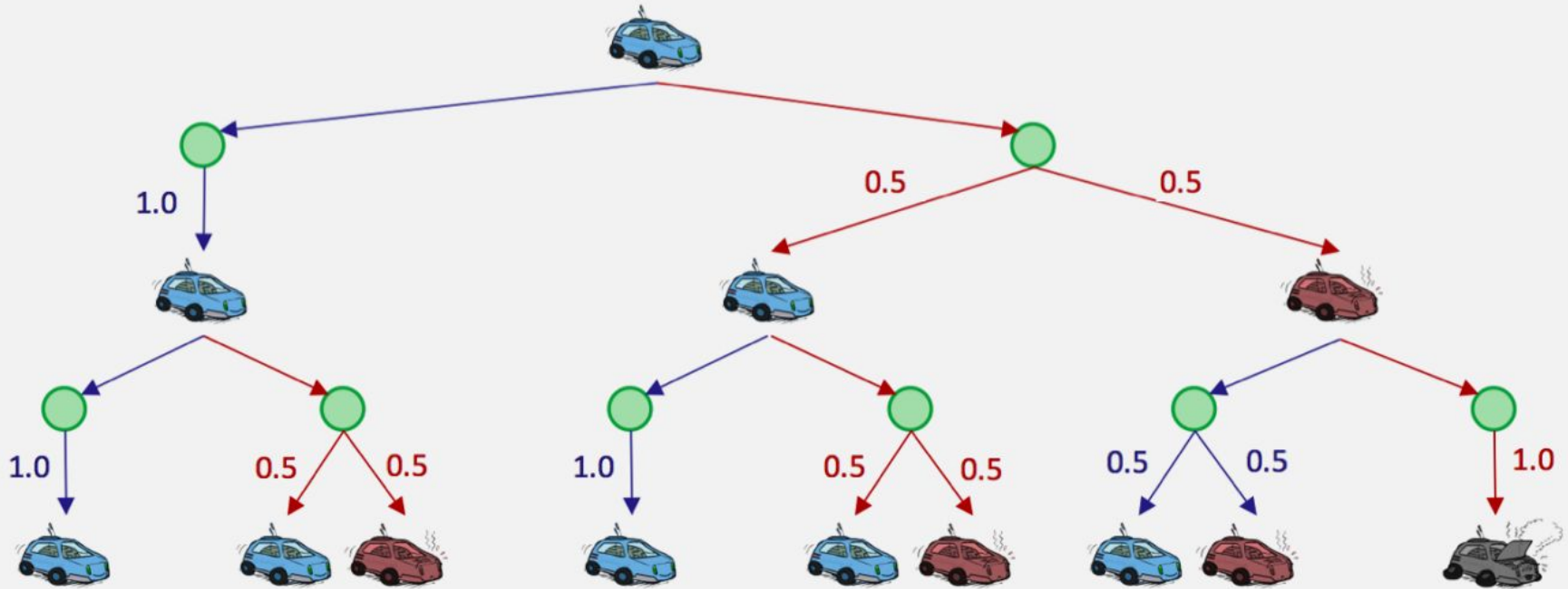
- $T(\text{cool}, \text{slow}, \text{cool}) = 1$
- $T(\text{warm}, \text{slow}, \text{cool}) = 0.5$
- $T(\text{warm}, \text{slow}, \text{warm}) = 0.5$
- $T(\text{cool}, \text{fast}, \text{cool}) = 0.5$
- $T(\text{cool}, \text{fast}, \text{warm}) = 0.5$
- $T(\text{warm}, \text{fast}, \text{overheated}) = 1$

- **Reward Function:** $R(s, a, s')$

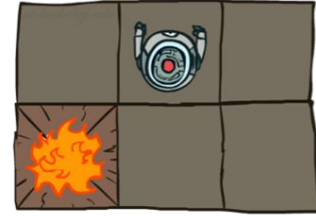
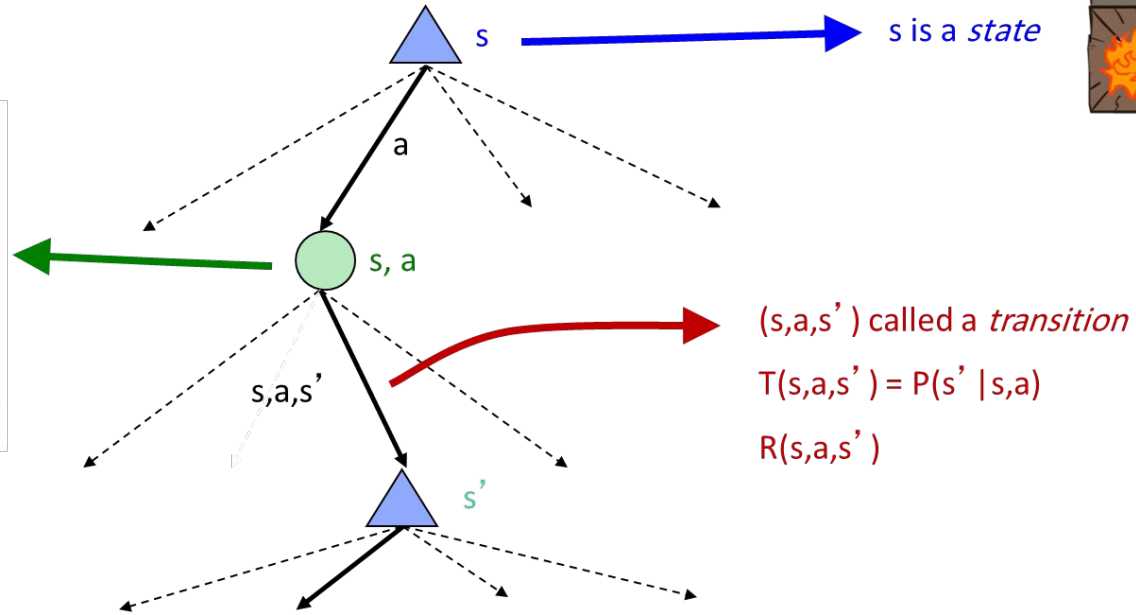
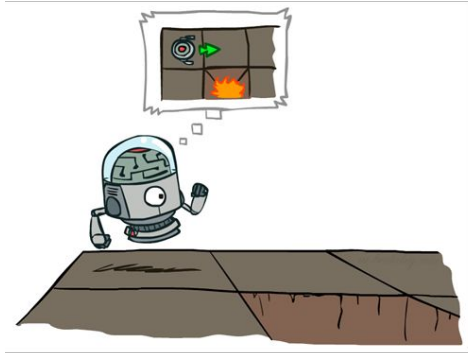
- $R(\text{cool}, \text{slow}, \text{cool}) = 1$
- $R(\text{warm}, \text{slow}, \text{cool}) = 1$
- $R(\text{warm}, \text{slow}, \text{warm}) = 1$
- $R(\text{cool}, \text{fast}, \text{cool}) = 2$
- $R(\text{cool}, \text{fast}, \text{warm}) = 2$
- $R(\text{warm}, \text{fast}, \text{overheated}) = -10$



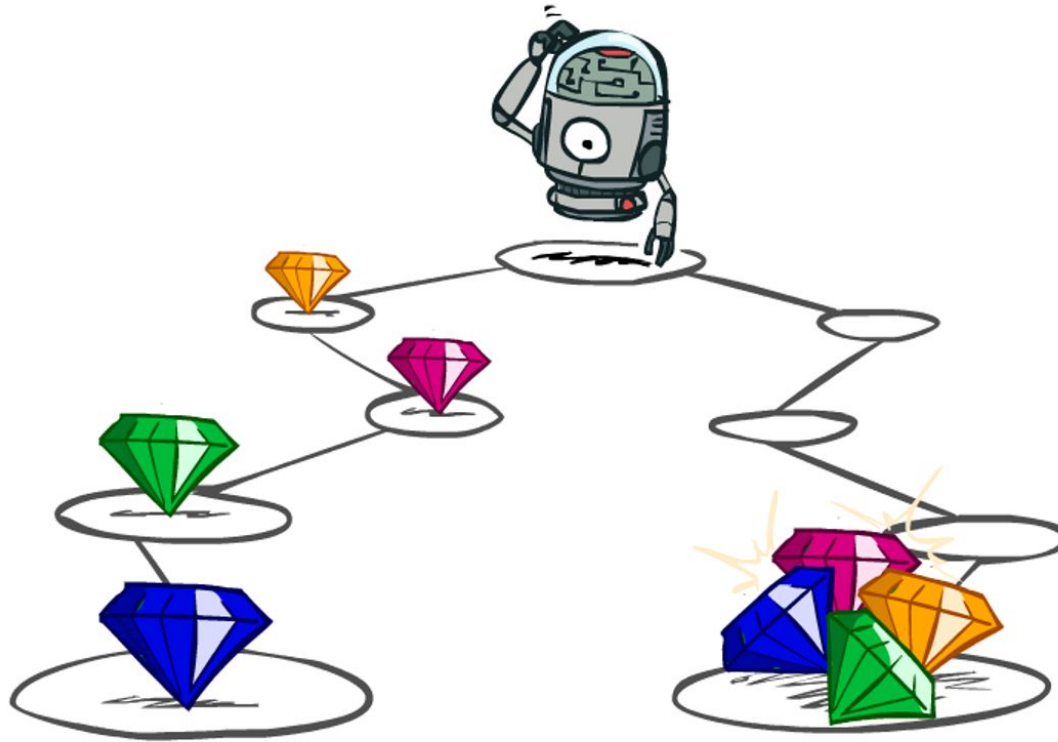
Racing Search Tree



MDP Search Trees



Utilities of Sequences

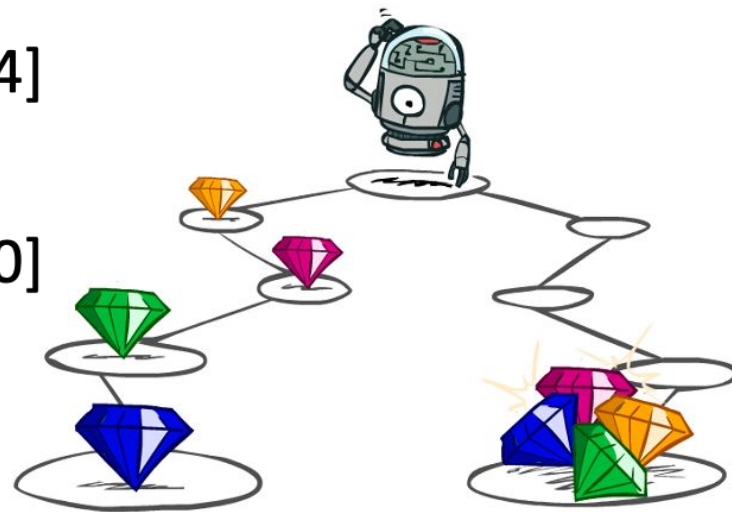


Utilities of Sequences

What preferences should an agent have over reward sequences?

More or less? $[1, 2, 2]$ or $[2, 3, 4]$

Now or later? $[0, 0, 1]$ or $[1, 0, 0]$



Discounting

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- One solution: values of rewards decay exponentially



1

Worth Now



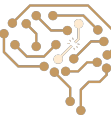
γ

Worth Next Step



γ^2

Worth In Two Steps



- How to discount?

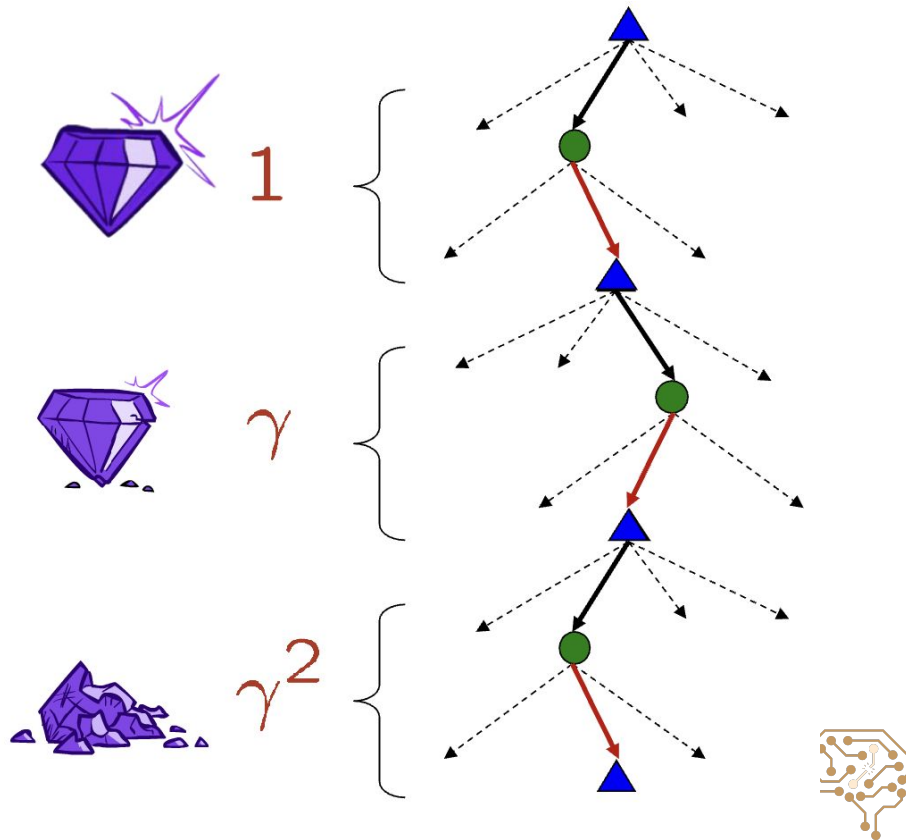
- Each time we descend a level, we multiply in the discount once

- Why discount?

- Sooner rewards probably do have higher utility than later rewards
- Also helps our algorithms converge

- Example: discount of 0.5

- $U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3$
- $U([1,2,3]) < U([3,2,1])$



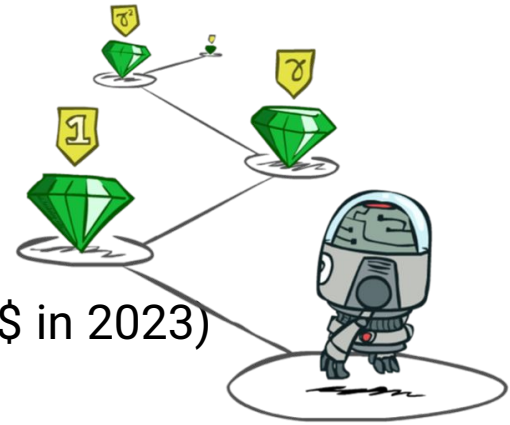
Finite Horizons and Discounting

► Problems with Utility Function:

- ☐ obtaining infinite reward?
- ☐ prefer rewards now to rewards later (1\$ in 2000 > 1\$ in 2023)

► Solutions:

- ☐ finite horizons: defines a "lifetime" for agents
- ☐ discount factors: additive utility -> discounted utility (get rid of infinite utility)



$$U([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$$



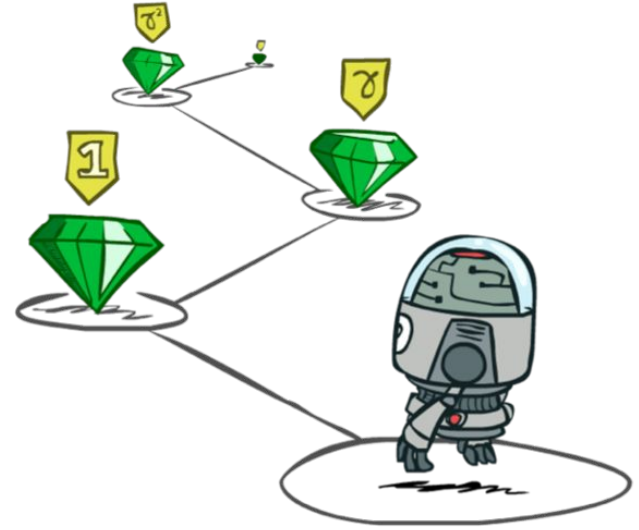
Stationary Preferences

Theorem: if we assume stationary preferences:

$$[a_1, a_2, \dots] \succ [b_1, b_2, \dots]$$



$$[r, a_1, a_2, \dots] \succ [r, b_1, b_2, \dots]$$



Then: there are only two ways to define utilities

Additive utility: $U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$

Discounted utility: $U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$



Stationary Preferences

- Given:

10				1
a	b	c	d	e

- Actions: East, West, and Exit (only available in exit states a, e)
- Transitions: deterministic

- Quiz 1: For $\gamma = 1$, what is the optimal policy?

10				1
----	--	--	--	---

- Quiz 2: For $\gamma = 0.1$, what is the optimal policy?

10				1
----	--	--	--	---

- Quiz 3: For which γ are West and East equally good when in state d?

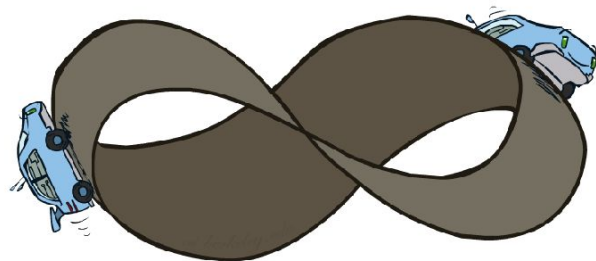


Infinite Utilities?!

- Problem: What if the game lasts forever? Do we get infinite rewards?

- Solutions:

- Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives nonstationary policies (π depends on time left)



- Discounting: use $0 < \gamma < 1$

$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\max} / (1 - \gamma)$$

- Smaller γ means smaller “horizon” – shorter term focus

- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like “overheated” for racing)



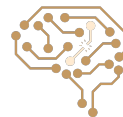
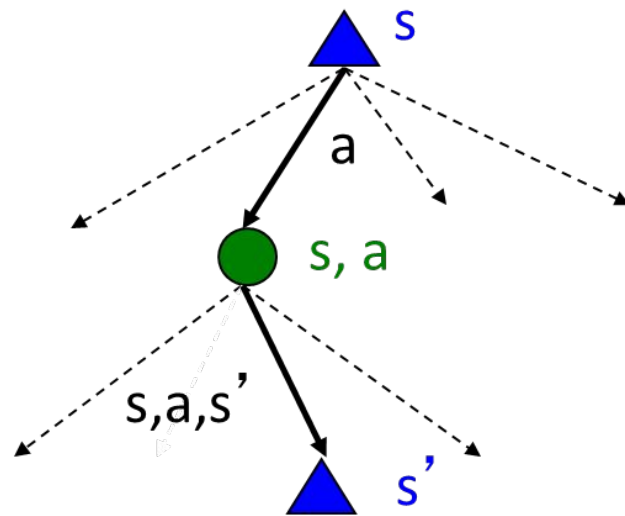
Recap: Defining MDPs

► Markov Decision Processes:

- Set of states S
- Set of actions A
- Transitions $P(s'|s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$

► MDP quantities so far:

- Policy = choice of action for each state
- Utility = sum of (discounted) rewards



Next Session: Solving MDPs

